Prüfen und Wiederholen

Parameter	Kursinformationen
Veranstaltung:	`Erhebung, Analyse und Visualisierung digitaler Daten`
Semester	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`if, elif, else, Einrückung, and/or/not, verschachtelte Bedingungen, for-Schleife, Zählmuster`
Link auf Repository:	https://github.com/TUBAF-IfI-LiaScript/VL_EAVD/blob/master/03_PruefenUndWiederholen.md
Autoren	Sebastian Zug & Bernhard Jung & Claude Code (Opus 5.5)

Leitfrage: *An wie vielen Tagen gab es 2024 auf dem Fichtelberg Frost?*

Fragen an die heutige Veranstaltung …

- Wie trifft ein Programm Entscheidungen?
- Warum ist Einrückung in Python keine Formsache?
- Wie verknüpft man mehrere Bedingungen, und wann schachtelt man sie ineinander?
- Wie wiederholt man dieselbe Anweisung für jeden Wert einer Messreihe?

> **Zum Nacharbeiten: die Ansicht „Lehrbuch“.** Im Hörsaal zeigen die Folien nur die Kernbotschaften. Die Erklärungen dazu stehen im Lehrbuch: oben rechts auf das Symbol mit den Schiebereglern klicken und „Lehrbuch“ wählen.

🗺️ Diese Vorlesung in der Kurslandkarte →

Heute lernt Ihr Programm zwei Dinge, die bisher gefehlt haben: entscheiden und wiederholen. Am Ende beantworten wir damit die Frage aus Vorlesung 01, diesmal mit 366 echten Werten. Der größte Stolperstein ist die Einrückung. Sie wird Ihnen heute einige Fehlermeldungen einbringen, das ist normal und gehört zum Lernen.

Rückblick

Aus der letzten Vorlesung wissen Sie: Ein Vergleich liefert einen Wahrheitswert.

```
1  minimum = -3.5
2  print(minimum < 0)
```

```
True
```

Die Frage aus Vorlesung 01 war aber nicht „Ist dieser Wert kleiner als 0?“, sondern:

> *Wie viele von 366 Werten sind kleiner als 0?*

Dafür brauchen wir zwei neue Werkzeuge: eines, das abhängig von einem Wahrheitswert **etwas tut oder lässt**, und eines, das dieselbe Prüfung **für jeden Wert wiederholt**.

`minimum < 0` liefert `True` oder `False`. Bisher konnte ein Programm diesen Wert nur ausgeben. Von heute an steuert er, was als Nächstes passiert. Aus Vorlesung 01 kennen Sie die drei Bausteine Folge, Verzweigung und Wiederholung. Die Folge haben Sie letzte Woche benutzt, die beiden anderen kommen heute dazu.

Dieselben Beispiele auf Ihrem Rechner

Alle Codebeispiele laufen hier direkt im Browser. Sie sind aber **gewöhnliches Python**: In einer Datei auf Ihrem Rechner ist das Ergebnis dasselbe.

Im Browser

```
1  minimum = -3.5
2  print("Minimum:", minimum)
3  print("Frost?", minimum < 0)
```

```
Another process is running, wait until finished
```

Auf Ihrem Rechner, als Datei `frost.py`:

```
$ python frost.py
Minimum: -3.5
Frost? True
```

> Nutzen Sie das: Jedes Beispiel dieser Vorlesung ist ein Ausgangspunkt für eigene Versuche in Visual Studio Code.

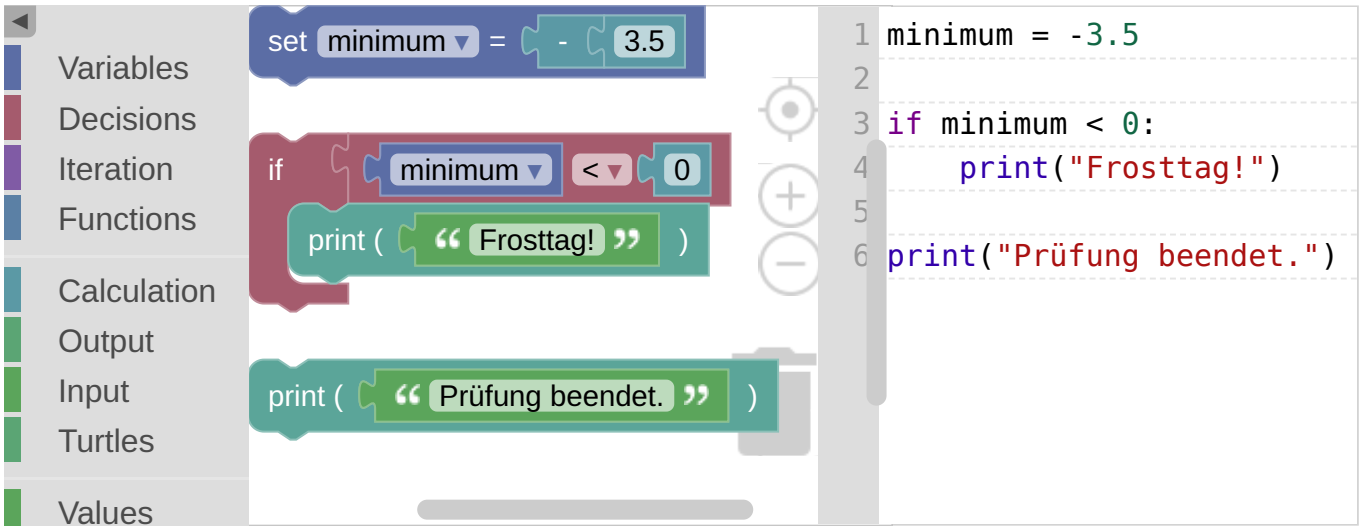
Es gibt eine einzige Ausnahme: Zeilen mit `open_url`, mit denen wir Daten aus unserem Repository, dem Ablageort der Kursdateien auf GitHub, in den Browser holen. Sie sind im Text jeweils markiert.

Das Dollarzeichen steht für die Eingabeaufforderung im Terminal und wird nicht mitgetippt. Unter macOS und Linux heißt der Befehl oft `python3`. Eine Datei können Sie speichern, verändern, erneut ausführen und weitergeben. Genau das unterscheidet ein Programm von einer Formel, die man einmal in eine Zelle tippt.

Entscheidungen: `if`

Bisher **meldet** das Programm nur, ob eine Bedingung zutrifft (`Frost? False`), und führt trotzdem **jede** Zeile aus. Zum Zählen darf das Programm aber nur an Frosttagen eins dazuzählen. Dafür brauchen wir eine **Verzweigung**: Der Wahrheitswert entscheidet, welche Zeilen laufen.

Ein Vergleich allein ist eine Feststellung, keine Handlung. Für einen einzelnen Wert reicht das: Wir lesen `True` oder `False` und entscheiden selbst. Bei 366 Werten soll das Programm entscheiden, und zwar bei jedem Wert neu: Frosttag, also mitzählen, sonst nichts tun. In der Tabellenkalkulation hat `ZÄHLENWENN` aus Vorlesung 01 diese Entscheidung versteckt erledigt. In Python schreiben wir sie selbst hin, und dieses Werkzeug heißt `if`.




```
Another process is running, wait until finished
```

Ändern Sie `minimum` im Textbereich auf `2.1`. Welche Zeile wird dann nicht mehr ausgegeben?

Lesen Sie die Zeile wie einen Satz: Wenn das Minimum kleiner als null ist, dann gib Frosttag aus. Hinter dem `if` steht immer etwas, das wahr oder falsch ist, hier der Vergleich aus der letzten Vorlesung. Ist es falsch, überspringt Python den eingerückten Teil und macht danach ganz normal weiter.

Einrückung ist Syntax

Woher weiß Python, welche Anweisungen zum `if` gehören? **An der Einrückung.** Sie gehört zur **Syntax**, also zu den Schreibregeln der Sprache. Die eingerückten Zeilen, die zusammengehören, heißen **Block**.

```
if minimum < 0:                  ← Doppelpunkt: jetzt kommt ein Block
    print("Frosttag!")           ← eingerückt: gehört zum if
    print("Unter null Grad.")    ← eingerückt: gehört zum if
print("Prüfung beendet.")        ← nicht eingerückt: läuft immer
```

Übliche Einrückung sind **vier Leerzeichen**. Visual Studio Code setzt sie automatisch, wenn Sie nach dem Doppelpunkt Enter drücken.

Zwei Programme, die sich nur in der Einrückung der letzten Zeile unterscheiden:

```
1  minimum = 2.1
2  if minimum < 0:
3      print("Frosttag!")
4  print("Minimum geprüft.")
```

```
Another process is running, wait until finished
```

```
1  minimum = 2.1
2  if minimum < 0:
3      print("Frosttag!")
4      print("Minimum geprüft.")
```

```
Another process is running, wait until finished
```

> In Python **ist** die Einrückung die Struktur. Falsch eingerückter Code ist falscher Code.

Die vier Leerzeichen sind eine Vereinbarung. Python verlangt nur, dass alle Zeilen eines Blocks gleich weit eingerückt sind.

In vielen anderen Sprachen markieren Klammern die Blöcke, die Einrückung ist dort nur Kosmetik. Python hat sich anders entschieden: Was für Menschen zusammengehörig aussieht, gehört auch für den Rechner zusammen. Beide Programme laufen übrigens fehlerfrei. Sie tun nur etwas anderes, und das meldet Python nicht.

Entweder – oder: `else`

Soll auch im anderen Fall etwas passieren, folgt ein `else`:

```
minimum = 2.1

if minimum < 0:
    print("Frosttag")
else:
    print("frostfrei")
```

```
Another process is running, wait until finished
```

Genau **einer** der beiden Blöcke wird ausgeführt — nie beide, nie keiner.

`else` hat keine eigene Bedingung, es steht für alle übrigen Fälle. Es gehört immer zu einem `if` und steht auf derselben Einrückungstiefe. Ein `if` ohne `else` ist erlaubt, ein `else` ohne `if` nicht.

Mehrere Fälle: `elif`

Der Deutsche Wetterdienst kennt feste Begriffe für besondere Tage. Anders als der Frosttag (Minimum unter 0 °C) sind diese über das **Tagesmaximum** definiert:

Begriff	Bedingung	Fichtelberg 2024
Eistag	Maximum < 0 °C	44
Sommertag	Maximum ≥ 25 °C	6
Heißer Tag	Maximum ≥ 30 °C	0

Mit `elif` („else if“) prüft man mehrere Fälle nacheinander:

```
maximum = 26.9

if maximum >= 30:
    print("Heißer Tag")
elif maximum >= 25:
    print("Sommertag")
elif maximum < 0:
    print("Eistag")
else:
    print("gewöhnlicher Tag")
```

```
Another process is running, wait until finished
```

Python prüft von oben nach unten und nimmt **den ersten Fall, der zutrifft**. Alle weiteren werden übersprungen.

Die Reihenfolge zählt. Was gibt dieses Programm für `maximum = 31.0` aus?

```
maximum = 31.0

if maximum >= 25:
    print("Sommertag")
elif maximum >= 30:
    print("Heißer Tag")
```

```
Another process is running, wait until finished
```

Legen Sie sich fest, **bevor** Sie das Programm ausführen:

- ◯ `Heißer Tag`
- ◯ `Sommertag`
- ◯ `Sommertag` und `Heißer Tag`

Am Frosttag fällt die Temperatur irgendwann unter null, am Eistag bleibt sie den ganzen Tag darunter. Jeder Eistag ist also auch ein Frosttag. `elif`-Zweige darf es beliebig viele geben, das `else` am Ende ist freiwillig. Ein Zweig muss auch nicht direkt etwas ausgeben. Oft merkt man sich das Ergebnis in einer Variablen, etwa `art = "Sommertag"`, und gibt es am Ende mit `print(art)` aus.

Bedingungen verknüpfen

Manchmal hängt eine Entscheidung von mehreren Bedingungen ab:

Operator	Bedeutung	Beispiel
`and`	beide müssen zutreffen	`minimum < 0 and maximum > 0`
`or`	mindestens eine muss zutreffen	`maximum >= 25 or minimum < -10`
`not`	kehrt den Wahrheitswert um	`not minimum < 0`

```
minimum = -2.4    # 2. Januar 2024
maximum = 2.6
```

```
4 if minimum < 0 and maximum > 0:
5     print("Frost-Tau-Wechsel: nachts Frost, tagsüber Tauwetter")
```

```
Another process is running, wait until finished
```

> Frost-Tau-Wechsel sind für Geologen und Bauingenieure interessant: Gefrierendes Wasser sprengt Gestein und Beton.

Der Fehlwert schlägt zurück

Erinnern Sie sich an `-999`? Der DWD kennzeichnet so Tage, an denen nicht gemessen wurde.

```
1 minimum = -999
2
3 if minimum < 0:
4     print("Frosttag")
```

```
Another process is running, wait until finished
```

Ein Tag ohne Messung als Frosttag gezählt — ohne jede Fehlermeldung. Korrigieren Sie die Bedingung.

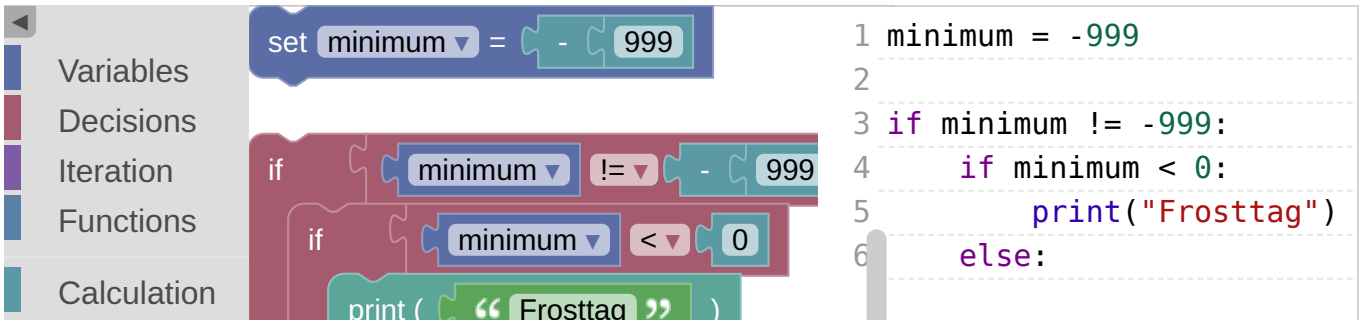
Lösung ›

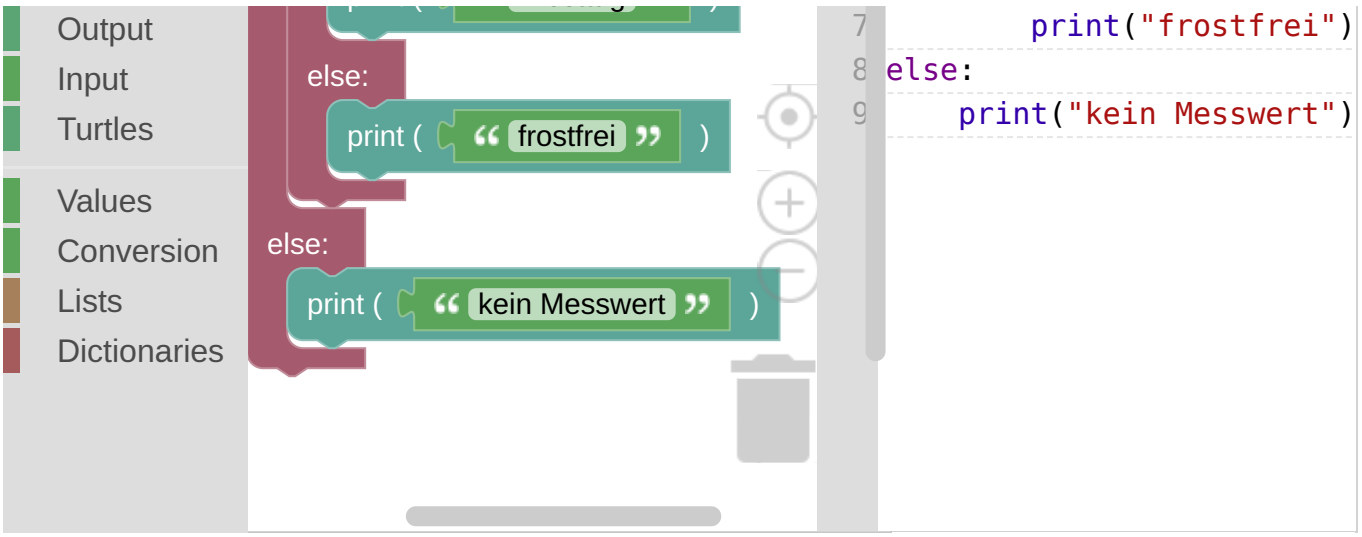
2024 gab es auf dem Fichtelberg 79 solcher Frost-Tau-Wechsel-Tage. `not minimum < 0` bedeutet dasselbe wie `minimum >= 0`. Python wertet zuerst `not` aus, dann `and`, zuletzt `or`. Wer unsicher ist, setzt runde Klammern.

Für Python ist `-999` eine ganz gewöhnliche Zahl, nur Sie wissen, dass sie „nicht gemessen“ bedeutet. Erinnern Sie sich an die -250,9 °C aus Vorlesung 00: Auch dort gab es keine Fehlermeldung.

Bedingung in der Bedingung

Ein `if` darf selbst wieder ein `if` enthalten. Das innere wird **nur geprüft, wenn das äußere zutrifft**. So rahmt die Fehlwert-Prüfung die Frostprüfung ein:






```
Another process is running, wait until finished
```

Setzen Sie `minimum` auf `-2.4` (2. Januar 2024) und auf `2.6` (1. März 2024): drei Werte, drei verschiedene Ausgaben.

Geht das nicht auch mit `and`? Ja, mit derselben Ausgabe für alle drei Werte:

```
minimum = -999

if minimum != -999 and minimum < 0:
    print("Frosttag")
elif minimum != -999:
    print("frostfrei")
else:
    print("kein Messwert")
```

```
Another process is running, wait until finished
```

Die Prüfung auf `-999` steht jetzt **zweimal** da. Wer sie ändern muss, etwa weil ein anderer Datensatz Fehlwerte als `-99` kennzeichnet, muss an zwei Stellen denken.

> **Faustregel:** `and`, wenn **ein** Fall beide Bedingungen braucht. **Verschachteln**, wenn eine Bedingung **mehrere** Fälle einrahmt, etwa „nur gemessene Werte auswerten“.

Sie wollen nur die Frosttage zählen, Fehlwerte sollen einfach übergangen werden. Was passt besser?

- ◯ `if minimum != -999 and minimum < 0:`
- ◯ ein `if` für den Fehlwert mit einem zweiten `if` darin

In den Blöcken sieht man die Verschachtelung sofort: Der innere `if`-Block steckt im äußeren wie eine Schachtel in der Schachtel. Im Text zeigt das allein die Einrückung, deshalb ist sie hier besonders wichtig. Das `else` in der vorletzten Zeile gehört zum äußeren `if`, weil es ganz links steht. Mehr als zwei, drei Ebenen werden schnell unübersichtlich. Dann lohnt es sich, die Bedingungen mit `and` zusammenzufassen oder, ab Vorlesung 07, Teile in Funktionen auszulagern.

Üben: Fehler finden bei `if`

⏱ **5 Minuten, zu zweit.** Jedes Programm enthält genau einen Fehler. Führen Sie es aus, reparieren Sie es direkt im Editor, bis die angegebene Ausgabe erscheint. Bei 4 und 5 hilft Ihnen keine Meldung.

1. Soll `Frosttag` ausgeben (2. Januar 2024):

```
1  minimum = -2.4
2  if minimum < 0
3      print("Frosttag")
```

```
Another process is running, wait until finished
```

2. Soll `kein Messwert` ausgeben:

```
1  minimum = -999
2  if minimum = -999:
3      print("kein Messwert")
```

```
Another process is running, wait until finished
```

3. Soll `frostfrei` ausgeben (1. März 2024):

```
1  minimum = 2.6
2  if minimum < 0:
3      print("Frosttag")
4  else:
5  print("frostfrei")
```

```
Another process is running, wait until finished
```

4. Soll `Eistag` ausgeben (1. Januar 2024, das Maximum blieb unter null):

```
1  minimum = -1.9
```

```
maximum = -0.7
if minimum < 0:
    print("Frosttag")
elif maximum < 0:
    print("Eistag")
```

```
Another process is running, wait until finished
```

5. Soll **nichts** ausgeben, denn am 1. März 2024 gab es keinen Frost:

```
minimum = 2.6
maximum = 7.3
if minimum < 0 or maximum > 0:
    print("Frost-Tau-Wechsel")
```

```
Another process is running, wait until finished
```

Lösung ›

Die ersten drei Fehler meldet Python, bevor auch nur eine Zeile läuft. Die beiden letzten laufen fehlerfrei und liefern trotzdem das Falsche. Solche **logischen Fehler** findet man nur, wenn man vorher weiß, was herauskommen soll. Deshalb steht bei jeder Aufgabe die erwartete Ausgabe, und deshalb prüfen wir Programme mit Tagen, deren Antwort wir kennen.

Wiederholen: `for`

Eine Prüfung für einen Wert können wir jetzt. Für 366 Werte brauchen wir eine **Schleife**.

In Python schreibt man mehrere Werte als **Liste** in eckige Klammern. Die `for`-Schleife nimmt sich dann **jeden Wert der Reihe nach** vor:

```
# Tiefsttemperaturen vom 8. bis 14. Januar 2024
minima = [-14.1, -12.1, -11.1, -10.2, -8.3, -7.5, -6.5]

for minimum in minima:
    print("Nächster Wert:", minimum)

print("Fertig.")
```

```
Another process is running, wait until finished
```

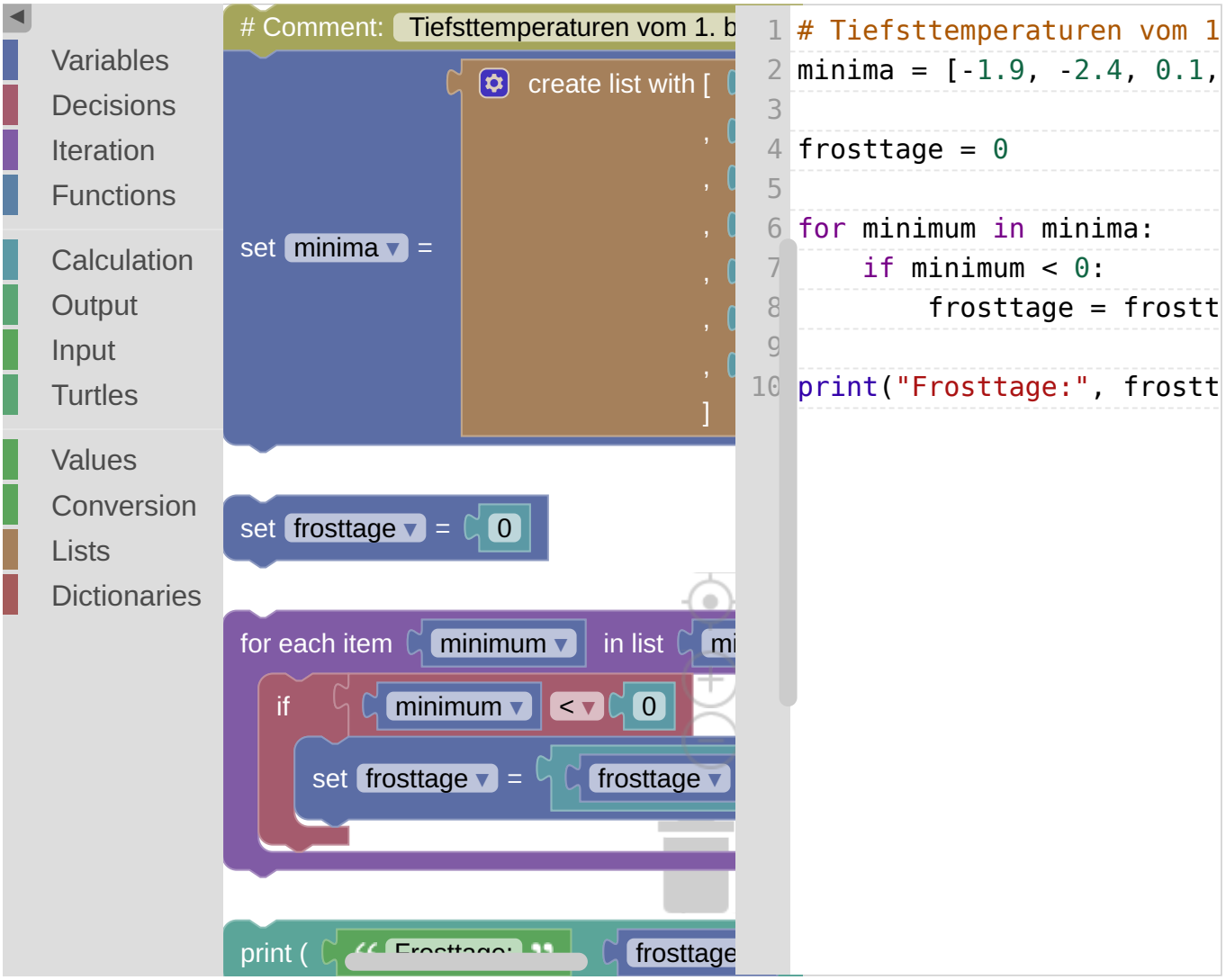
Lesen Sie die Schleife als: *„Für jedes `minimum` in `minima`: führe den eingerückten Block aus.“*

> Listen lernen Sie in Vorlesung 05 genauer kennen. Heute reicht: Eine Liste enthält mehrere Werte, und `for` geht sie der Reihe nach durch.

Der Name `minimum` bekommt bei jedem Durchlauf den nächsten Wert der Liste. Sie wählen ihn selbst, `for m in minima:` täte dasselbe. Wenn Sie in der Tabellenkalkulation eine Formel nach unten ziehen, wiederholen Sie auch dieselbe Anweisung für jede Zeile. Die Schleife schreibt diese Wiederholung einmal auf, statt sie in jede Zeile zu kopieren.

Das Zählmuster

Jetzt setzen wir alles zusammen — Schleife, Bedingung und eine Variable, die mitzählt:




```
Another process is running, wait until finished
```

Dieses **Zählmuster** hat immer drei Teile:

Teil	Code	Wo?
Zähler anlegen	`frosttage = 0`	**vor** der Schleife
Bedingt erhöhen	`frosttage = frosttage + 1`	**in** der Schleife, im `if`
Ergebnis nutzen	`print(...)`	**nach** der Schleife

In der Tabellenkalkulation übernimmt `ZÄHLENWENN` das Zählen, hier machen Sie es selbst. Die Variable `frosttage` ist das Gedächtnis des Programms: Sie ist die Antwort auf die Frage aus Vorlesung 02, was man sich merken muss. In Vorlesung 04 kommen weitere solche Muster dazu.

Spielen wir die Logik auch noch mal durch!

Durchlauf	`minimum`	`minimum < 0`	`frosttage` danach
–	–	–	0
1	-1.9	True	1
2	-2.4	True	2
3	0.1	False	2
4	-1.7	True	3
5	-1.7	True	4
6	-4.2	True	5
7	-11.9	True	6

Die Tabelle ist ein Ablaufprotokoll wie in den letzten Vorlesungen, diesmal mit einer Zeile pro Durchlauf. Achten Sie auf Durchlauf 3: Die Bedingung ist falsch, der Zähler bleibt stehen, die Schleife läuft trotzdem weiter. Die erste Zeile ohne Durchlauf ist `frosttage = 0`. Sie wird genau einmal ausgeführt, vor der Schleife.

Der Algorithmus aus Vorlesung 01

In Vorlesung 01 haben wir das Zählen als Pseudocode aufgeschrieben. Hier steht er Zeile für Zeile in Python, wie versprochen in **fünf Zeilen**:

Pseudocode (Vorlesung 01)

```
Setze frosttage auf 0.
Für jedes Minimum in der Liste:
    Wenn Minimum < 0:
        Erhöhe frosttage um 1.
Gib frosttage aus.
```

Python

```python
frosttage = 0
for minimum in minima:
    if minimum < 0:
        frosttage = frosttage + 1
print("Frosttage:", frosttage)
```

> Wer den Pseudocode aufgeschrieben hat, hat die eigentliche Arbeit schon getan. Python verlangt nur noch die genaue Schreibweise.

Typische Fehler

Was geben diese Varianten aus — und warum?

```python
minima = [-1.9, -2.4, 0.1]
for minimum in minima:
    frosttage = 0
    if minimum < 0:
        frosttage = frosttage + 1
print(frosttage)
```

```
0
```

```python
minima = [-1.9, -2.4, 0.1]
frosttage = 0
for minimum in minima:
    if minimum < 0:
        frosttage = frosttage + 1
    print(frosttage)
```

```
```

```
1
2
2
```

Lösung ›

Die beiden Varianten sehen fast gleich aus, jeweils steht nur eine Zeile an einer anderen Stelle. Führen Sie sie im Kopf aus, bevor Sie sie starten. Dass ein Programm durchläuft, heißt auch hier nicht, dass es die gesuchte Zahl liefert.

Typische Fehlvorstellung: Die Schleifenvariable

Typische Fehlvorstellung: *„Nach der Schleife ist die Schleifenvariable weg — oder sie steht wieder am Anfang."*

```python
minima = [-1.9, -2.4, 0.1, -1.7]

for minimum in minima:
    print("in der Schleife:", minimum)

print("nach der Schleife:", minimum)
```

```
Another process is running, wait until finished
```

Legen Sie sich fest, **bevor** Sie das Programm ausführen: Was gibt die letzte Zeile aus?

- ◯ eine Fehlermeldung, weil `minimum` nur in der Schleife existiert
- ◯ `nach der Schleife: -1.9`
- ◯ `nach der Schleife: -1.7`

In einigen anderen Sprachen, etwa Java oder C, ist eine Zählvariable, die im Schleifenkopf angelegt wird, nach der Schleife tatsächlich nicht mehr bekannt. Wollen Sie einen Wert über die Schleife hinaus behalten, legen Sie dafür am besten vor der Schleife eine eigene Variable an.

Fehlermeldungen bei `if` und `for`

Mit `if` und `for` kommen drei typische Meldungen dazu. Sie zeigen meist direkt auf die richtige Stelle, anders als der Tippfehler aus Vorlesung 02, und oft schlägt Python die Korrektur gleich mit vor.

Im Code	Meldung (letzte Zeile)	Korrektur
`if minimum < 0`	`SyntaxError: expected ':'`	Doppelpunkt ans Zeilenende
`if minimum = 0:`	`SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?`	vergleichen mit `==`
Block nicht eingerückt	`IndentationError: expected an indented block after 'if' statement on line 4`	vier Leerzeichen einrücken

`:=` aus der zweiten Meldung brauchen Sie nicht, diesen Vorschlag können Sie ignorieren.

Bei einem Syntaxfehler läuft gar nichts, auch nicht die Zeilen davor: Python prüft zuerst das ganze Programm. Das ergänzt das „Zeile für Zeile“ aus Vorlesung 02. Ein `NameError` oder `TypeError` taucht dagegen erst auf, wenn die Zeile an der Reihe ist. Die Zeilen davor sind dann schon gelaufen.

Ein Fehler nach dem anderen

Python meldet immer nur den ersten Fehler. Nach jeder Korrektur erneut ausführen, bis das Programm durchläuft.

Das Programm soll die Frosttage zählen. Führen Sie es aus, korrigieren Sie die gemeldete Stelle, führen Sie es erneut aus — bis `Frosttage: 2` erscheint. Wie viele Fehler waren es?

```
1  minima = [-1.9, -2.4, 0.1]
2  frosttage = 0
3  for minimum in minima
4      if minimum < 0:
5      frosttage = frosttage + 1
6  print("Frosttage:", frosttage)
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
          ~~~~~~~~~~^
        source,
        ^^^^^^^
    ...<5 lines>...
        optimize=optimize,
        ^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/lib/python313.zip/_pyodide/_base.py", line 285, in __init__
    self.ast = next(self._gen)
               ~~~~^^^^^^^^^^^
  File "/lib/python313.zip/_pyodide/_base.py", line 149, in
_parse_and_compile_gen
    mod = compile(source, filename, mode, flags | ast.PyCF_ONLY_AST)
  File "<exec>", line 3
    for minimum in minima
                         ^
SyntaxError: expected ':'
```

Lösung ›

Dass nach der ersten Korrektur ein neuer Fehler erscheint, wirkt am Anfang entmutigend. Es ist aber ein Fortschritt: Python ist eine Zeile weiter gekommen. Bei längeren Programmen hilft es, nach jeder kleinen Änderung auszuführen, statt zwanzig Zeilen zu schreiben und dann alle Fehler auf einmal zu suchen.

Üben: Fehler finden bei `for`

⏱ **5 Minuten, zu zweit.** Alle vier Programme sollen `Frosttage: 3` ausgeben. Nur eines meldet einen Fehler. Bei den anderen hilft ein Ablaufprotokoll: Was steht nach jedem Durchlauf in `frosttage`?

1. Am dritten Tag fehlt die Messung:

```python
minima = [-1.9, -2.4, -999, -1.7]
frosttage = 0
for minimum in minima:
    if minimum < 0:
        frosttage = frosttage + 1
print("Frosttage:", frosttage)
```

```
Frosttage: 4
```

2.

```
1  minima = [-1.9, -2.4, 0.1, -1.7]
2  frosttage = 0
3  for minimum in minima:
4      if minimum < 0:
5          frosttage = frosttage + minimum
6  print("Frosttage:", frosttage)
```

```
Frosttage: -6.0
```

3.

```
1  minima = [-1.9, -2.4, 0.1, -1.7]
2  frosttage = 0
3  for minimum in minima:
4      if minima < 0:
5          frosttage = frosttage + 1
6  print("Frosttage:", frosttage)
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 4, in <module>
TypeError: '<' not supported between instances of 'list' and 'int'
```

4.

```
1  minima = [-1.9, -2.4, 0.1, -1.7]
2  frosttage = 0
3  for minimum in minima:
4      if minimum < 0:
5          frosttage + 1
6  print("Frosttage:", frosttage)
```

```
Frosttage: 0
```

Lösung ›

Drei der vier Programme laufen ohne Meldung und liefern trotzdem eine falsche Zahl. Ein Ablaufprotokoll mit vier Zeilen genügt, um sie zu finden. Programm 2 zeigt den Unterschied zwischen Zählen und Summieren: In Vorlesung 04 brauchen wir genau dieses Summieren, um einen Mittelwert zu berechnen. Der dritte Fehler ist ein Klassiker, weil `minimum` und `minima` fast gleich aussehen. Wählen Sie Namen, die sich deutlich unterscheiden, oder lesen Sie die Meldung genau: `list` verrät, dass hier eine ganze Liste verglichen wird.

Die Antwort auf die Leitfrage

Die 366 Tiefsttemperaturen des Jahres 2024 liegen in unserem Repository, ein Wert pro Zeile. Statt sie abzutippen, holen wir sie direkt von dort. Quelle: Deutscher Wetterdienst, Station 01358, Lizenz CC BY 4.0.

```
from pyodide.http import open_url

url = ("https://raw.githubusercontent.com/TUBAF-IfI-LiaScript/"
       "VL_EAVD/master/data/fichtelberg/2024_minimum.txt")
text = open_url(url).read()

frosttage = 0
for wert in text.split():
    if float(wert) < 0:
        frosttage = frosttage + 1

print("Frosttage 2024:", frosttage)
```

```
Another process is running, wait until finished
```

127 Frosttage, dieselbe Zahl wie mit `ZÄHLENWENN` in Vorlesung 01. Zwei unabhängige Wege, ein Ergebnis: So prüft man ein Programm.

Das Zählmuster kennen Sie. Neu sind nur die Zeilen, die die Daten holen und zerlegen. **Sie müssen sie heute nicht im Detail verstehen:** `open_url(url).read()` holt die Datei als einen langen Text, `text.split()` zerlegt ihn in einzelne Werte.

`from pyodide.http import open_url` stellt die Funktion `open_url` bereit. Die Adresse steht in runden Klammern, damit sie auf zwei Zeilen passt. `text.split()` zerlegt den Text an Zeilenumbrüchen und Leerzeichen: Aus `"-1.9\n-2.4\n0.1"` wird die Liste `["-1.9", "-2.4", "0.1"]`, dabei steht `\n` für einen Zeilenumbruch. Das sind immer noch Texte. Erst `float(wert)` macht daraus Zahlen, genau wie in Vorlesung 02. Wie man Dateien liest, behandeln wir ausführlich in Vorlesung 06.

Nur im Browser

`pyodide.http.open_url` gibt es nur in der Python-Umgebung dieser Webseite. In Visual Studio Code auf Ihrem Rechner führt die Zeile zu einem `ModuleNotFoundError`. Dort lesen Sie die Datei direkt — wie das geht, zeigt Vorlesung 06.

Das Zählmuster wird nicht länger, wenn die Daten mehr werden: Dieselben Zeilen zählen 7 oder 366 Werte. **Aber Vorsicht:** 2024 enthält keinen einzigen Fehlwert, deshalb stimmt die 127. Auf alle 47.595 Tage seit 1890 angewandt, käme dieselbe Bedingung auf 22.330 statt 22.320 Frosttage, denn zehn Tage stehen auf `-999`. Eine Meldung gäbe es nicht.

Erweitern Sie das Programm

1. Zählen Sie zusätzlich die Tage mit **strengem Frost** (Minimum unter -10 °C).
2. Wie viele Tage waren **frostfrei**? Lösen Sie das auf zwei Wegen: mit einem zweiten Zähler und ohne.
3. Zählen Sie die Tage, an denen das Minimum **genau** 0,0 °C war. Welcher Vergleichsoperator ist dafür nötig?

Zur Kontrolle ›

Kontrollfragen

Was gibt dieses Programm aus?

```
werte = [3, -1, 4, -1, -5]
a = 0
b = 0
for w in werte:
    if w < 0:
        a = a + 1
    else:
        b = b + w
print(a, b)
```

Am 5. Januar 2024 lag das Maximum auf dem Fichtelberg bei genau 0,0 °C. Welche Aussagen stimmen?

- [ ] Mit der Bedingung `maximum < 0` ist der 5. Januar **kein** Eistag.
- [ ] Mit der Bedingung `maximum <= 0` ist der 5. Januar **kein** Eistag.
- [ ] Die Wahl zwischen `<` und `<=` ändert das Ergebnis, wenn Werte genau auf der Grenze liegen.
- [ ] Bei Kommazahlen spielt der Unterschied zwischen `<` und `<=` keine Rolle.

Welche Bedingung erkennt einen Frosttag, ohne dass Fehlwerte (`-999`) mitgezählt werden?

- ◯ `minimum < 0 or minimum != -999`
- ◯ `minimum < 0 and minimum != -999`
- ◯ `minimum < 0 and minimum == -999`
- ◯ `not minimum == -999`

Ordnen Sie zu: Was steht vor, in oder nach der Schleife?

vor	in	nach	
◯	◯	◯	`frosttage = 0`
◯	◯	◯	`if minimum < 0:`
◯	◯	◯	`frosttage = frosttage + 1`
◯	◯	◯	`print("Frosttage:", frosttage)`

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester. Wir schlagen die Begriffe vor, Sie ergänzen, was fehlt: Aus Ihren Datenwünschen in Vorlesung 00 sind so schon die ersten Anwendungen geworden. Nehmen Sie sich dafür jetzt fünf Minuten.

Welches Wort von heute müssten Sie jemandem aus Ihrem Semester erklären können?

Texteingabe …

Welche Daten aus Ihrem Fach würden Sie gern so auswerten?

Texteingabe …

Nächste Woche

Das Zählmuster merkt sich eine Zahl über alle Durchläufe hinweg. In der nächsten Vorlesung merken wir uns mehr — und beantworten eine Frage, an der die Tabellenkalkulation verzweifelt:

> *Wie lang war die längste ununterbrochene Frostperiode?*

Zur Vorbereitung

- [ ] Schreiben Sie `frost.py` auf Ihrem Rechner mit den sieben Januarwerten und bringen Sie es zum Laufen.
- [ ] Überlegen Sie in Pseudocode: Was muss man sich merken, um die längste Frostperiode zu finden?
- [ ] Wie würden Sie den **Mittelwert** der sieben Werte mit einer Schleife berechnen?

Zur Übung

Blatt 3 (Erdbeben einordnen) steht nach der Vorlesung in OPAL. Versuchen Sie sich **vor** der Übung an den Aufgaben, gern auch zu zweit. Besprochen wird es in der Übung ab dem 16.11.